# Task 5 Code Optimization

## Prompt :

Optimize the following Python code by reducing its time complexity. Explain the original complexity, the optimized approach, and provide the improved code with comments.

Code :

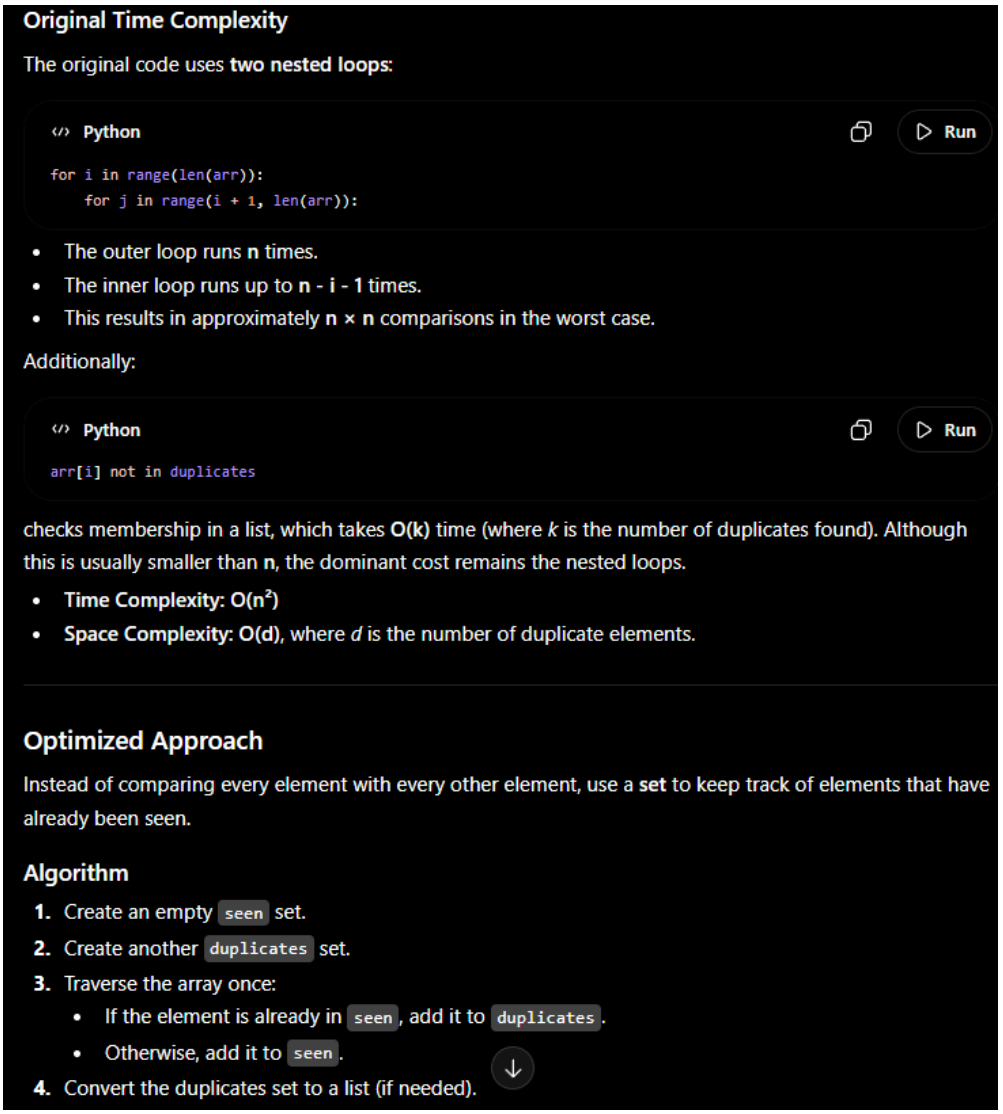
```
arr = [1, 2, 3, 2, 4, 1]

duplicates = []

for i in range(len(arr)):
    for j in range(i + 1, len(arr)):
        if arr[i] == arr[j] and arr[i] not in duplicates:
            duplicates.append(arr[i])

print(duplicates)
```

Output :



Since **set lookup and insertion are O(1) on average**, the entire algorithm requires only one pass through the array.

- **Time Complexity: O(n)**
- **Space Complexity: O(n)**

---

## Optimized Code

Python Run

```python
arr = [1, 2, 3, 2, 4, 1]

seen = set()         # Stores elements we've already encountered
duplicates = set()   # Stores duplicate elements

for num in arr:
    if num in seen:
        duplicates.add(num)   # Add duplicate to the set
    else:
        seen.add(num)         # First occurrence of the element

# Convert to list if a list output is required
print(list(duplicates))
```

### Output

Python Run

```
[1, 2]
```

> **Note:** Since `duplicates` is a set, the order of elements is not guaranteed. If preserving the order of first duplicate occurrence is important, use the following version:

Python Run

```python
arr = [1, 2, 3, 2, 4, 1]

seen = set()
duplicates = []

for num in arr:
    if num in seen:
        if num not in duplicates:
            duplicates.append(num)
    else:
        seen.add(num)

print(duplicates)
```

### Output:

Python Run

```
[2, 1]
```

This version preserves the order in which duplicates are first detected while still avoiding the expensive nested loops. Although checking `num not in duplicates` is a list lookup, it is performed only when a duplicate is encountered, making the overall approach much more efficient than the original nested-loop solution in typical cases.